PROPOSAL PROYEK PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
SEMESTER GENAP 2025/2

Judul Projek	: SiKePang: Sistem Ketahanan Pangan	
Bidang Tema	: Ketahanan Pangan	
Kelompok	: 5	
Anggota Kelompok	: Zakhwa Aliya Maryam	(152024032)
	: Putri Yudi Patrecia	(152024035)
	: Zeta Mardhotillah Ronny	(152024047)
	: Dzakiyya Puteri Aulia	(152024127)

Dosen Pembimbing	: Uung Ungkawa
Asisten Lab	: Fida, Dindin, Riza

LABORATORIUM SINC

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

INSTITUT TEKNOLOGI NASIONAL

BANDUNG

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penyusunan proposal proyek "SiKePang: Sistem Ketahanan Pangan" ini dapat dilaksanakan dengan baik. Proposal ini disusun sebagai bagian dari pemenuhan Tugas Besar Praktikum Pemrograman Berorientasi Objek, serta sebagai bentuk implementasi penerapan konsep *Object-Oriented Programming* (OOP) menggunakan teknologi Spring Boot dan Java 17+.

Penyusunan proyek ini bertujuan untuk menghadirkan solusi digitalisasi pengelolaan data ketahanan pangan di tingkat komunitas yang lebih efektif dan terintegrasi. Dengan membangun sistem backend yang mencakup manajemen data petani, pencatatan komoditas, pengelolaan stok pangan, hingga pengelolaan distribusi pangan, diharapkan proyek ini dapat memberikan gambaran nyata penerapan teknologi informasi untuk memecahkan permasalahan pencatatan manual dalam ketersediaan pangan masyarakat.

Penulis menyadari bahwa proposal ini masih memiliki keterbatasan dan kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan di masa yang akan datang. Semoga rancangan Sistem Ketahanan Pangan ini dapat memberikan manfaat, baik bagi penulis maupun pembaca, serta dapat menjadi referensi dalam pembelajaran implementasi OOP yang komprehensif menggunakan framework Spring Boot modern.

Bandung, 26 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR **1**
DAFTAR ISI **2**
BAB I PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Rumusan Masalah 3
1.3 Tujuan Sistem 4
1.4 Batasan Sistem 4
BAB II LANDASAN TEORI **6**
2.1 Konsep Object-Oriented Programming (OOP) 6
2.1.1 Encapsulation (Enkapsulasi) 6
BAB III PERANCANGAN SISTEM **10**
3.1 Desain Database 10
BAB IV RENCANA IMPLEMENTASI **20**
4.1 Lingkungan Pengembangan (Development Environment) 20
4.2 Tahapan Implementasi 20
4.3 Pembagian Tugas Kelompok 22
BAB V KESIMPULAN **23**
DAFTAR PUSTAKA **24**

BAB I PENDAHULUAN

1.1 Latar Belakang

Ketahanan pangan merupakan salah satu pilar fundamental pembangunan nasional dan kesejahteraan masyarakat. Berdasarkan Undang-Undang No. 18 Tahun 2012 tentang Pangan, ketahanan pangan didefinisikan sebagai kondisi terpenuhinya pangan bagi negara sampai dengan perseorangan, yang tercermin dari tersedianya pangan yang cukup, baik jumlah maupun mutunya, aman, beragam, bergizi, merata, dan terjangkau serta tidak bertentangan dengan agama, keyakinan, dan budaya masyarakat.

Indonesia sebagai negara agraris memiliki potensi pertanian yang sangat besar. Namun demikian, pengelolaan data pertanian dan distribusi pangan masih banyak dilakukan secara konvensional dan manual. Hal ini menyebabkan berbagai permasalahan seperti ketidakakuratan data stok pangan, keterlambatan distribusi, sulitnya pemantauan ketersediaan pangan secara real-time, hingga minimnya informasi yang dapat diakses oleh pemangku kepentingan.

Di tingkat komunitas, kelompok tani sering kali kesulitan dalam mencatat dan melaporkan hasil panen, mengelola stok komoditas pangan, serta memantau proses distribusi pangan kepada masyarakat. Pencatatan yang masih manual rentan terhadap kesalahan data dan sulit untuk diakses secara bersamaan oleh berbagai pihak yang berkepentingan.

Oleh karena itu, dibutuhkan sebuah sistem informasi digital yang dapat mengelola data ketahanan pangan secara terintegrasi, mulai dari manajemen data petani dan kelompok tani, pencatatan komoditas, pengelolaan stok pangan, hingga distribusi pangan kepada masyarakat. Sistem ini dibangun berbasis teknologi Spring Boot dengan konsep *Object Oriented Programming* (OOP) yang terstruktur, sehingga mudah dikembangkan, dipelihara, dan diintegrasikan dengan sistem lain di masa mendatang.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, berikut adalah rumusan masalah yang akan diselesaikan oleh sistem ini:

a. Bagaimana cara mengelola data petani dan kelompok tani secara digital agar mudah diakses dan diperbarui?
b. Bagaimana cara mencatat, memantau, dan mengelola data komoditas pangan beserta stok yang tersedia secara akurat dan real-time?

c. Bagaimana proses distribusi pangan dapat dipantau dan dicatat secara sistematis sehingga dapat dipertanggungjawabkan?
d. Bagaimana membangun sistem backend yang menerapkan prinsip OOP secara penuh (Encapsulation, Inheritance, Polymorphism, Abstraction) dengan arsitektur yang terstruktur dan dapat diakses publik?

1.3 Tujuan Sistem

Tujuan dari pembangunan Sistem Ketahanan Pangan ini adalah:

a. Membangun sistem backend berbasis Spring Boot yang dapat mengelola data petani, kelompok tani, dan komoditas pangan secara digital.
b. Menyediakan fitur pencatatan dan pemantauan stok pangan secara real-time untuk memudahkan pengambilan keputusan.
c. Menyediakan fitur manajemen distribusi pangan yang terstruktur, lengkap dengan riwayat distribusi ke berbagai tujuan.
d. Menyediakan endpoint REST API yang dapat diakses publik melalui Cloudflare Tunnel untuk integrasi dengan aplikasi front-end atau sistem lain.
e. Menerapkan konsep OOP secara penuh meliputi Encapsulation, Inheritance, Polymorphism, dan Abstraction dengan arsitektur berlapis (Layered Architecture).

1.4 Batasan Sistem

Untuk menjaga fokus dan kelayakan pengerjaan, sistem ini memiliki batasan-batasan sebagai berikut:

- Yang DIKERJAKAN dalam sistem ini:
    a. Manajemen data pengguna (admin, petugas, petani) dengan autentikasi berbasis role.
    b. Manajemen data petani dan kelompok tani (pendataan, pembaruan, penghapusan).
    c. Manajemen data komoditas pangan beserta kategori dan satuan.
    d. Pengelolaan stok pangan: pencatatan stok masuk, stok keluar, dan saldo stok terkini.
    e. Manajemen distribusi pangan: pencatatan distribusi beserta detail komoditas yang didistribusikan.
    f. Monitoring ketersediaan pangan: laporan stok per komoditas dan riwayat distribusi.

    g. REST API yang dapat diakses publik via Cloudflare Tunnel.
- Yang TIDAK DIKERJAKAN dalam sistem ini:
    a. Sistem pembayaran atau transaksi keuangan.
    b. Notifikasi real-time (push notification, email, SMS).
    c. Integrasi dengan sistem pemerintah atau BULOG secara langsung.
    d. Pengelolaan logistik dan pengiriman fisik.

BAB II LANDASAN TEORI

2.1 Konsep Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) adalah paradigma pemrograman yang mengorganisasikan perangkat lunak sebagai kumpulan objek yang saling berinteraksi. Setiap objek merupakan instans dari sebuah class yang memiliki atribut (data) dan method (perilaku). OOP memiliki empat pilar utama yang wajib diterapkan dalam sistem ini:

2.1.1 Encapsulation (Enkapsulasi)

Encapsulation adalah mekanisme pembungkusan data (atribut) dan method yang memanipulasinya ke dalam satu unit (class), serta membatasi akses langsung ke komponen internal suatu objek. Dalam implementasi, seluruh atribut class dideklarasikan sebagai private dan hanya dapat diakses melalui getter dan setter yang bersifat public. Hal ini bertujuan untuk melindungi integritas data dari modifikasi yang tidak sah.

Contoh penerapan dalam sistem ini: Class Petani memiliki atribut private seperti idPetani, namaKelompok, alamat, luas_lahan, dan hanya dapat diakses melalui method getPetani() dan setPetani() yang dikontrol dengan validasi.

2.1.2 Inheritance (Pewarisan)

Inheritance adalah mekanisme di mana sebuah class (subclass/child class) mewarisi atribut dan method dari class lain (superclass/parent class). Inheritance memungkinkan penggunaan kembali kode (code reuse) dan membangun hierarki class yang logis sesuai domain masalah.

Contoh penerapan dalam sistem ini: Class Petani dan class Admin sama-sama mewarisi class abstrak BasePengguna yang berisi atribut umum seperti id, nama, email, password, dan createdAt. Dengan demikian, kode tidak perlu ditulis ulang untuk setiap class.

2.1.3 Polymorphism (Polimorfisme)

Polymorphism adalah kemampuan sebuah objek untuk mengambil banyak bentuk. Dalam OOP, polymorphism umumnya diwujudkan melalui method overriding (subclass mendefinisikan ulang method superclass) dan method overloading. Dengan

polymorphism, sebuah referensi dari superclass dapat merujuk ke objek subclass dan memanggil method yang telah di-override.

Contoh penerapan dalam sistem ini: Interface PetaniService mendefinisikan kontrak method seperti getAllPetani(), getPetaniById(), dll. Class PetaniServiceImpl mengimplementasikan interface tersebut dan memberikan implementasi konkret. Saat controller memanggil method melalui referensi PetaniService, Java secara polimorfis memanggil implementasi yang benar di PetaniServiceImpl.

2.1.4 Abstraction (Abstraksi)

Abstraction adalah proses menyembunyikan detail implementasi dan hanya menampilkan fungsionalitas yang relevan kepada pengguna. Abstraksi diwujudkan melalui abstract class dan interface. Abstract class dapat memiliki atribut, method konkret, dan method abstrak yang wajib diimplementasikan oleh subclass. Interface mendefinisikan kontrak (daftar method) yang harus dipenuhi oleh class yang mengimplementasikannya.

Contoh penerapan dalam sistem ini: Class BasePengguna dideklarasikan sebagai abstract class yang memuat atribut dan method umum, sekaligus mendefinisikan method abstrak getRole() yang wajib diimplementasikan oleh subclass (Petani, Admin).

2.2 Teknologi yang Digunakan

2.2.1 Java 17+ (LTS)

Java 17 adalah versi Long-Term Support (LTS) dari bahasa pemrograman Java yang dipublikasikan oleh Oracle. Java 17 membawa berbagai peningkatan performa dan fitur baru seperti sealed classes, pattern matching, dan enhanced switch expressions. Java dipilih karena matang, memiliki ekosistem library yang luas, dan mendukung penuh paradigma OOP.

2.2.2 Spring Boot

Spring Boot adalah framework berbasis Java yang menyederhanakan pengembangan aplikasi enterprise dengan konfigurasi minimal. Spring Boot menyediakan berbagai fitur bawaan (auto-configuration, embedded server, starter dependencies) sehingga pengembang dapat fokus pada logika bisnis. Spring Boot digunakan sebagai framework utama untuk membangun REST API di sistem ini.

2.2.3 Spring Data JPA

Spring Data JPA adalah bagian dari ekosistem Spring yang menyediakan abstraksi atas Java Persistence API (JPA) untuk operasi database. Dengan Spring Data JPA, pengembang cukup mendefinisikan interface repository yang meng-extend JpaRepository, dan framework secara otomatis menyediakan implementasi CRUD. Spring Data JPA juga mendukung query method berdasarkan nama method dan anotasi @Query untuk query kustom.

2.2.4 REST API

REST (Representational State Transfer) adalah arsitektur komunikasi berbasis HTTP yang menggunakan prinsip stateless, uniform interface, dan resource-based. API RESTful menggunakan HTTP methods seperti GET (membaca data), POST (membuat data), PUT/PATCH (memperbarui data), dan DELETE (menghapus data). Setiap resource direpresentasikan oleh URL yang unik dan data biasanya dipertukarkan dalam format JSON.

2.2.5 MySQL

MySQL adalah sistem manajemen basis data relasional (RDBMS) open-source yang paling banyak digunakan. MySQL mendukung SQL standar, transaksional ACID, dan integrasi yang baik dengan Spring Data JPA melalui driver JDBC. Database ini digunakan untuk menyimpan seluruh data sistem ketahanan pangan secara persisten.

2.3 Teknologi Deployment

2.3.1 Cloudflare Tunnel

Cloudflare Tunnel (sebelumnya Argo Tunnel) adalah layanan yang memungkinkan server lokal dapat diakses dari internet publik tanpa perlu membuka port pada firewall atau memiliki IP publik statis. Cloudflare Tunnel membuat koneksi keluar dari server ke jaringan Cloudflare, sehingga request dari internet dapat diteruskan ke server lokal secara aman melalui enkripsi TLS. Layanan ini menjadi solusi deployment yang wajib digunakan dalam proyek ini.

2.3.2 Mini PC (Proxmox) / STB (Debian)

Deployment dilakukan pada salah satu dari dua opsi infrastruktur yang disediakan laboratorium. Opsi A menggunakan Mini PC berbasis Proxmox yang

merupakan platform virtualisasi open-source berbasis Debian, memungkinkan pengelolaan Virtual Machine (VM) dan Container (LXC) secara fleksibel. Opsi B menggunakan Set-Top Box (STB) yang menjalankan OS Debian sebagai edge device deployment. Kedua opsi ini memungkinkan aplikasi Spring Boot berjalan aktif dan diakses melalui URL publik via Cloudflare Tunnel.

2.3.3 Layered Architecture

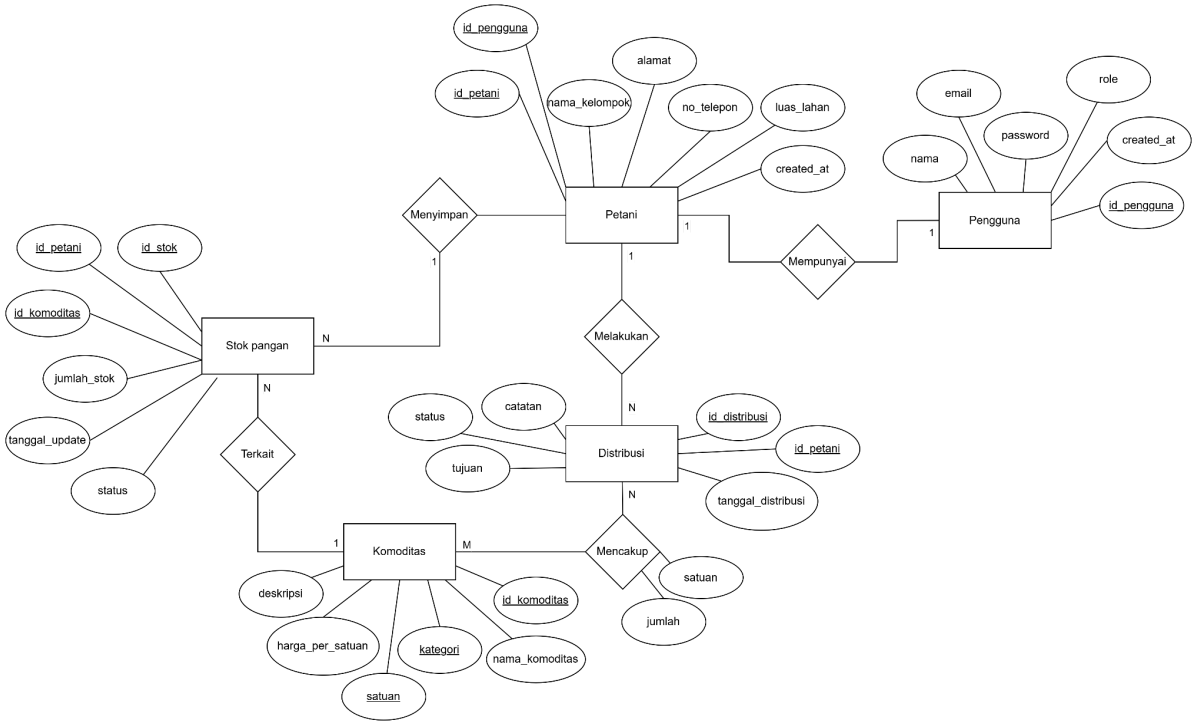
Sistem dibangun menggunakan Layered Architecture yang membagi aplikasi menjadi lapisan-lapisan terpisah dengan tanggung jawab yang jelas. Arsitektur ini terdiri dari: (1) Controller Layer yang menerima request HTTP dan mengembalikan response JSON; (2) Service Layer yang berisi logika bisnis dan didefinisikan melalui interface beserta implementasinya; (3) Repository Layer yang menangani akses data ke database; dan (4) Model/Entity Layer yang merepresentasikan struktur data domain.

BAB III PERANCANGAN SISTEM

3.1 Desain Database

Database sistem dirancang menggunakan pendekatan relasional dengan MySQL. Desain ini memenuhi seluruh syarat minimal yang ditetapkan: minimal 4 tabel, minimal 2 relasi one-to-many, minimal 1 relasi many-to-many, setiap tabel memiliki primary key yang jelas, dan foreign key antar tabel dijelaskan.

3.1.1 Entity Relationship Diagram (ERD)



3.1.2 Daftar Tabel dan Atribut

Tabel 1: pengguna

Nama Kolom	Tipe Data	Constraint	Keterangan
id_pengguna	BIGINT	PK, AUTO_INCREMENT	Primary key unik
nama	VARCHAR(100)	NOT NULL	Nama lengkap pengguna
email	VARCHAR(150)	NOT NULL, UNIQUE	Email login unik

password	VARCHAR(255)	NOT NULL	Password terenkripsi (BCrypt)
role	ENUM	NOT NULL	ADMIN / PETUGAS / PETANI
created_at	TIMESTAMP	DEFAULT NOW()	Waktu akun dibuat

Tabel 2: petani

Nama Kolom	Tipe Data	Constraint	Keterangan
id_petani	BIGINT	PK, AUTO_INCREMENT	Primary key unik
id_pengguna	BIGINT	FK -> pengguna, UNIQUE	Referensi akun pengguna (one-to-one)
nama_kelompok	VARCHAR(100)	NOT NULL	Nama kelompok tani
alamat	TEXT	NOT NULL	Alamat lengkap kelompok tani
no_telepon	VARCHAR(20)	NULL	Nomor kontak penanggung jawab
luas_lahan	DECIMAL(10,2)	NULL	Luas lahan dalam hektar
created_at	TIMESTAMP	DEFAULT NOW()	Waktu data dibuat

Tabel 3: komoditas

Nama Kolom	Tipe Data	Constraint	Keterangan
id_komoditas	BIGINT	PK, AUTO_INCREMENT	Primary key unik
nama_komoditas	VARCHAR(100)	NOT NULL, UNIQUE	Nama komoditas (padi, jagung, dll)
kategori	VARCHAR(50)	NOT NULL	Kategori (serealia, sayuran, buah, dll)

satuan	VARCHAR(20)	NOT NULL	Satuan ukur (kg, ton, kuintal)
harga_per_satuan	DECIMAL(15,2)	NOT NULL	Harga referensi per satuan
deskripsi	TEXT	NULL	Keterangan tambahan komoditas

Tabel 4: stok_pangan

Nama Kolom	Tipe Data	Constraint	Keterangan
id_stok	BIGINT	PK, AUTO_INCREMENT	Primary key unik
id_petani	BIGINT	FK -> petani, NOT NULL	Referensi petani pemilik stok
id_komoditas	BIGINT	FK -> komoditas, NOT NULL	Referensi komoditas
jumlah_stok	DECIMAL(15,2)	NOT NULL	Jumlah stok tersedia
tanggal_update	DATE	NOT NULL	Tanggal pencatatan stok
status	ENUM	DEFAULT 'TERSEDIA'	TERSEDIA / MENIPIS / HABIS

Tabel 5: distribusi

Nama Kolom	Tipe Data	Constraint	Keterangan
id_distribusi	BIGINT	PK, AUTO_INCREMENT	Primary key unik
id_petani	BIGINT	FK -> petani, NOT NULL	Referensi petani pengirim
tanggal_distribusi	DATE	NOT NULL	Tanggal pelaksanaan distribusi
tujuan	VARCHAR(200)	NOT NULL	Lokasi atau instansi tujuan

status	ENUM	DEFAULT 'PROSES'	PROSES / SELESAI / BATAL
catatan	TEXT	NULL	Catatan tambahan distribusi

Tabel 6: distribusi_komoditas (Junction Table — Many-to-Many)

Nama Kolom	Tipe Data	Constraint	Keterangan
id	BIGINT	PK, AUTO_INCREMENT	Primary key unik
id_distribusi	BIGINT	FK -> distribusi, NOT NULL	Referensi distribusi
id_komoditas	BIGINT	FK -> komoditas, NOT NULL	Referensi komoditas
jumlah	DECIMAL(15,2)	NOT NULL	Jumlah komoditas didistribusikan
satuan	VARCHAR(20)	NOT NULL	Satuan (kg, ton, dsb)

3.1.3 Relasi Antar Tabel

Tabel Asal	Tipe Relasi	Tabel Tujuan	Penjelasan
pengguna	One-to-One	petani	Satu akun pengguna dapat menjadi satu data petani (id_pengguna di tabel petani bersifat UNIQUE)
petani	One-to-Many	stok_pangan	Satu petani dapat memiliki banyak data stok pangan yang dicatat
komoditas	One-to-Many	stok_pangan	Satu jenis komoditas dapat muncul di banyak data stok dari berbagai petani
petani	One-to-Many	distribusi	Satu petani dapat melakukan banyak distribusi pangan ke berbagai tujuan

distribusi	Many-to-Many	komoditas	Satu distribusi dapat memuat banyak komoditas; satu komoditas dapat ada di banyak distribusi. Diwujudkan melalui tabel distribusi_komoditas

3.2 Desain OOP & Rancangan Class Diagram

Sistem dirancang menggunakan prinsip OOP secara menyeluruh dengan Layered Architecture: Controller -> Service (Interface + Impl) -> Repository -> Entity/DTO. Berikut adalah class yang dirancang beserta penjelasan hubungan antar class.

3.2.1 Daftar Class dan Peran

Nama Class	Layer/Anotasi	Tipe	Keterangan
BasePengguna	Model	Abstract Class	Class abstrak induk yang mendefinisikan atribut umum (id, nama, email, password, createdAt) dan method abstrak getRole()
Petani	@Entity	Class (extends BasePengguna)	Mewarisi BasePengguna, menambah atribut namaKelompok, alamat, noTelepon, luasLahan. Mengimplementasikan getRole() -> 'PETANI'
Admin	@Entity	Class (extends BasePengguna)	Mewarisi BasePengguna, menambah atribut divisi. Mengimplementasikan getRole() -> 'ADMIN'
Komoditas	@Entity	Class	Entity komoditas pangan dengan atribut namaKomoditas, kategori, satuan, hargaPerSatuan, deskripsi
StokPangan	@Entity	Class	Entity stok pangan, asosiasi ke Petani dan Komoditas (ManyToOne). Atribut:

			jumlahStok, tanggalUpdate, status
Distribusi	@Entity	Class	Entity distribusi, asosiasi ke Petani (ManyToOne) dan ke DistribusiKomoditas (OneToMany)
DistribusiKomoditas	@Entity	Class	Junction entity untuk relasi many-to-many antara Distribusi dan Komoditas. Atribut tambahan: jumlah, satuan
PetaniService	Service	Interface	Mendefinisikan kontrak: getAllPetani(), getPetaniById(), savePetani(), deletePetani(), findByKelompok()
PetaniServiceImpl	@Service	Class (implements PetaniService)	Implementasi konkret dari PetaniService, berisi logika bisnis manajemen data petani
KomoditasService	Service	Interface	Mendefinisikan kontrak: getAllKomoditas(), getById(), save(), delete(), findByKategori()
KomoditasServiceImpl	@Service	Class (implements)	Implementasi konkret dari KomoditasService
StokPanganService	Service	Interface	Mendefinisikan kontrak: getAllStok(), getStokByPetani(), addStok(), updateStok(), deleteStok()
StokPanganServiceImpl	@Service	Class (implements)	Implementasi konkret dari StokPanganService, termasuk logika perhitungan status stok
DistribusiService	Service	Interface	Mendefinisikan kontrak: getAllDistribusi(), getById(), createDistribusi(), updateStatus(), deleteDistribusi()
DistribusiServiceImpl	@Service	Class (implements)	Implementasi konkret dari DistribusiService

PetaniController	**@RestController**	**Class**	**Menerima HTTP request untuk resource /api/petani, mendelegasikan ke PetaniService**
KomoditasController	**@RestController**	**Class**	**Menerima HTTP request untuk resource /api/komoditas**
StokPanganController	**@RestController**	**Class**	**Menerima HTTP request untuk resource /api/stok-pangan**
DistribusiController	**@RestController**	**Class**	**Menerima HTTP request untuk resource /api/distribusi**
PetaniRepository	**@Repository**	**Interface (extends JpaRepository)**	**Akses database untuk entity Petani. Memiliki custom query findByNamaKelompokContaining()**
KomoditasRepository	**@Repository**	**Interface (extends JpaRepository)**	**Akses database untuk entity Komoditas**
StokPanganRepository	**@Repository**	**Interface (extends JpaRepository)**	**Akses database untuk entity StokPangan, memiliki query findByPetaniIdPetani()**
DistribusiRepository	**@Repository**	**Interface (extends JpaRepository)**	**Akses database untuk entity Distribusi, query findByStatus()**

3.2.2 Penjelasan Hubungan Antar Class

A. Inheritance (Pewarisan)

BasePengguna (abstract class) <-- Petani (class). Class Petani meng-extend BasePengguna, mewarisi atribut id, nama, email, password, createdAt, serta wajib mengimplementasikan method abstrak getRole() yang mengembalikan string 'PETANI'. Hal yang sama berlaku untuk class Admin yang mengembalikan 'ADMIN'. Ini bukan sekadar extends JpaRepository, melainkan hierarki class domain yang bermakna.

B. Association (Asosiasi)

1. StokPangan berasosiasi dengan Petani (ManyToOne): banyak stok dapat dimiliki satu petani.
2. StokPangan berasosiasi dengan Komoditas (ManyToOne): banyak stok dapat merujuk satu komoditas.
3. Distribusi berasosiasi dengan Petani (ManyToOne): banyak distribusi dapat dilakukan satu petani.
4. Distribusi berasosiasi dengan DistribusiKomoditas (OneToMany): satu distribusi dapat memiliki banyak detail komoditas.
5. DistribusiKomoditas berasosiasi dengan Komoditas (ManyToOne): merealisasikan relasi many-to-many antara Distribusi dan Komoditas.

C. Interface & Composition (Service Layer)

Setiap Service mendefinisikan interface (kontrak) dan diimplementasikan oleh class Impl. Controller tidak langsung menginstansiasi ServiceImpl, melainkan mendeklarasikan dependency bertipe interface (e.g., PetaniService petaniService) yang di-inject oleh Spring IoC Container (@Autowired / constructor injection). Inilah realisasi dari Dependency Inversion Principle dan Polymorphism.

D. Encapsulation

Seluruh atribut pada semua class (entity maupun DTO) dideklarasikan private. Akses dilakukan melalui getter dan setter public dengan validasi internal (misalnya setter jumlahStok memvalidasi nilai tidak boleh negatif). Anotasi Lombok @Getter @Setter dapat digunakan untuk mempersingkat kode.

3.3 Rancangan REST API

Seluruh endpoint REST API dirancang sesuai konvensi RESTful dengan Base URL: https:///api. Setiap endpoint dilengkapi dengan HTTP method, URL, parameter, dan deskripsi singkat.

3.3.1 Endpoint Manajemen Petani (/api/petani)

Method	URL	Parameter	Deskripsi

GET	/api/petani	-	Mengambil daftar semua petani beserta informasi kelompok tani
GET	/api/petani/{id}	id (Path): Long	Mengambil detail data petani berdasarkan ID. Melempar ResourceNotFoundException jika tidak ditemukan
GET	/api/petani/cari	keyword (Query): String	Mencari petani berdasarkan nama kelompok tani (case-insensitive, partial match)
POST	/api/petani	Body: PetaniRequest (JSON)	Menambahkan data petani baru. Validasi: nama, email, alamat wajib diisi
PUT	/api/petani/{id}	id (Path): Long + Body: JSON	Memperbarui seluruh data petani berdasarkan ID
DELETE	/api/petani/{id}	id (Path): Long	Menghapus data petani berdasarkan ID. Soft delete atau hard delete sesuai konfigurasi

3.3.2 Endpoint Manajemen Komoditas (/api/komoditas)

Method	URL	Parameter	Deskripsi
GET	/api/komoditas	-	Mengambil semua data komoditas pangan yang terdaftar
GET	/api/komoditas/{id}	id (Path): Long	Mengambil detail komoditas berdasarkan ID spesifik
GET	/api/komoditas/kategori	kategori (Query): String	Filter komoditas berdasarkan kategori (serealia, sayuran, buah, dll)
POST	/api/komoditas	Body: KomoditasRequest (JSON)	Menambahkan komoditas pangan baru ke sistem
PUT	/api/komoditas/{id}	id (Path): Long + Body	Memperbarui informasi komoditas berdasarkan ID

DELETE	/api/komoditas/{id}	id (Path): Long	Menghapus komoditas dari sistem berdasarkan ID

3.3.3 Endpoint Manajemen Stok Pangan (/api/stok-pangan)

Method	URL	Parameter	Deskripsi
GET	/api/stok-pangan	-	Mengambil semua data stok pangan dari seluruh petani
GET	/api/stok-pangan/{id}	id (Path): Long	Mengambil detail stok pangan berdasarkan ID
GET	/api/stok-pangan/petani/{idPetani}	idPetani (Path): Long	Mengambil semua stok pangan milik satu petani spesifik
POST	/api/stok-pangan	Body: StokPanganRequest (JSON)	Menambahkan pencatatan stok pangan baru (idPetani, idKomoditas, jumlahStok, tanggalUpdate wajib)
PUT	/api/stok-pangan/{id}	id (Path): Long + Body	Memperbarui jumlah stok dan status ketersediaan berdasarkan ID
DELETE	/api/stok-pangan/{id}	id (Path): Long	Menghapus data stok pangan berdasarkan ID

3.3.4 Endpoint Manajemen Distribusi & Monitoring (/api/distribusi, /api/monitoring)

Method	URL	Parameter	Deskripsi
GET	/api/distribusi	-	Mengambil semua data distribusi pangan dari seluruh petani
GET	/api/distribusi/{id}	id (Path): Long	Mengambil detail satu data distribusi beserta daftar

			komoditas yang didistribusikan
POST	/api/distribusi	Body: DistribusiRequest (JSON)	Membuat data distribusi baru beserta detail komoditas yang didistribusikan (nested request body)
PUT	/api/distribusi/{id}/status	id (Path) + status (Body)	Memperbarui status distribusi (PROSES -> SELESAI atau PROSES -> BATAL)
DELETE	/api/distribusi/{id}	id (Path): Long	Menghapus data distribusi (hanya bisa jika status masih PROSES)
GET	/api/monitoring/ketersediaan	-	Dashboard monitoring: ringkasan total stok per komoditas, status ketersediaan, dan riwayat distribusi terbaru

BAB IV RENCANA IMPLEMENTASI

4.1 Lingkungan Pengembangan (Development Environment)

Komponen	Detail
IDE	IntelliJ IDEA Community / VS Code + Spring Boot Extension Pack
JDK	Java Development Kit 17 LTS (Eclipse Temurin / Oracle JDK)
Build Tool	Apache Maven 3.8+ (via Spring Initializr)
Database Local	MySQL 8.0 (via XAMPP / Docker Container)
API Testing	Postman / Insomnia untuk pengujian endpoint REST API
Version Control	Git + GitHub (setiap anggota bekerja di branch masing-masing)
Deployment Server	Mini PC berbasis Proxmox (Opsi A) atau STB Debian (Opsi B)

4.2 Tahapan Implementasi

Fase	Kegiatan	Keterangan	Alokasi waktu
1	Setup & Inisialisasi	Buat project Spring Boot via Spring Initializr, konfigurasi dependency (Spring Web, Spring Data JPA, MySQL Driver, Lombok, Validation)	Minggu 7, P7
2	Implementasi Model & Database	Buat class entity (BasePengguna, Petani, Admin, Komoditas, StokPangan, Distribusi, DistribusiKomoditas), konfigurasi JPA anotasi, jalankan migrasi database	Minggu 7-8, P7/P8

3	Repository Layer	Buat interface repository extends JpaRepository, definisikan custom query method	Minggu 8, P8
4	Service Layer	Buat interface service dan implementasinya, terapkan logika bisnis, exception handling, validasi data	Minggu 8-9, P8/P9
5	Controller Layer	Buat RestController, mapping endpoint sesuai rancangan API, buat DTO request/response, konfigurasi CORS	Minggu 9, P9
6	Exception Handling	Buat custom exception (ResourceNotFoundException , ValidationException) dan GlobalExceptionHandler dengan @ControllerAdvice	Minggu 9, P9
7	Testing & Debugging	Uji seluruh endpoint menggunakan Postman, perbaiki bug, validasi response format JSON	Minggu 10, P10
8	Deployment	Deploy ke server Proxmox/STB, setup Cloudflare Tunnel, pastikan URL publik dapat diakses dari luar jaringan	Minggu 10-11, P10/P11

4.3 Pembagian Tugas Kelompok

Anggota	Tanggung Jawab Utama	Keterangan
1 (Ketua)	Backend Core + Koordinasi	Setup project, implementasi entity & repository, koordinasi integrasi antar modul
2	Service Layer Petani & Komoditas	Implementasi PetaniService, KomoditasService beserta Impl, custom exception
3	Service Layer Stok & Distribusi	Implementasi StokPanganService, DistribusiService beserta Impl, logika monitoring
4	Controller Layer + Deployment	Implementasi seluruh REST Controller, DTO, konfigurasi Cloudflare Tunnel

BAB V KESIMPULAN

Proposal ini telah menguraikan rancangan lengkap “SiKePang: Sistem Ketahanan Pangan” berbasis Spring Boot dan Java 17+ yang akan dibangun sebagai Tugas Besar Praktikum Pemrograman Berorientasi Objek. Sistem ini dirancang untuk mendigitalisasi pengelolaan data ketahanan pangan di tingkat komunitas, mulai dari manajemen data petani, pencatatan komoditas, pengelolaan stok pangan, hingga distribusi pangan.

Rancangan sistem ini telah memenuhi seluruh ketentuan dan syarat minimal yang ditetapkan, meliputi:

1. Desain database dengan 6 tabel, 3 relasi one-to-many, dan 1 relasi many-to-many
2. Desain OOP dengan lebih dari 20 class, penerapan abstract class (BasePengguna), inheritance (Petani dan Admin extends BasePengguna), polymorphism (Service interface + implementation), encapsulation (private attributes + getter/setter), dan layered architecture (Controller -> Service -> Repository)
3. Rancangan 23 REST API endpoint yang mencakup semua HTTP method yang dibutuhkan.

Sistem ini diharapkan dapat menjadi solusi digital yang efektif untuk mendukung ketahanan pangan masyarakat dan menjadi media pembelajaran implementasi OOP yang komprehensif menggunakan teknologi Spring Boot modern.

DAFTAR PUSTAKA

Bloch, J. Effective Java, 3rd Edition. Addison-Wesley, 2018.

Cloudflare Tunnel Documentation. https://developers.cloudflare.com/cloudflareone/connections/connect-networks/. Diakses April 2026.

Craig Walls. Spring in Action, 6th Edition. Manning Publications, 2022.

MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/. Diakses April 2026.

Spring Framework Documentation. https://spring.io/projects/spring-boot. Diakses April 2026.

Undang-Undang Republik Indonesia No. 18 Tahun 2012 tentang Pangan.